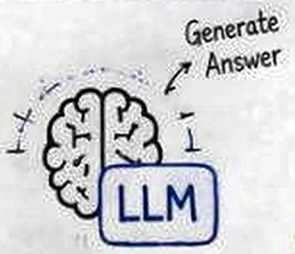


300 RAG Interview Questions & Answers

Documents

Retrieval (RAG)

- Search
- Similarity
- Rank
- Filter
- Rerank

Relevant Context

Grounded Answer

Reasoning
RAG
Tool Use
Grounded Responses

A Complete Interview Handbook

RAG Pipeline (High Level)

Documents → **Ingestion & Processing**
- Load
- Chunk
- Embed
- Store Metadata

→ **Vector DB (Index)**
- Similarity Search
- Hybrid Search
- Filtering
- Reranking

→ Retrieve Relevant Chunks → LLM (Generate) → Grounded Answer

What's Inside ?

1. RAG Fundamentals (Q1 – Q20)
2. Document Processing & Chunking (Q21 – Q40)
3. Embeddings & Vector Databases (Q41 – Q60)
4. Retrieval Techniques (Q61 – Q80)
5. Advanced RAG Architectures (Q81 – Q100)
6. GraphRAG, Agentic RAG & Multimodal RAG (Q101 – Q180)
7. RAG Evaluation & Metrics (Q121 – Q140)
8. Debugging, Observability & Error Analysis (Q141 – Q160)
9. Performance, Scaling & Cost Optimization (Q161 – Q180)
10. RAG Security & Guardrails (Q181 – Q200)
11. Production Deployment & System Design (Q201 – Q280)
12. Seenario-Based RAG Questions (Q221 – Q240)
13. Real World Use Cases (Q241 – Q260)
14. Best Practices & Design Patterns (Q261 – Q280)
15. Miscellaneous & Advanced Topics (Q281 – Q300)

Key Features

- [x] 300 most important interview questions
- [x] Detailed, concise and easy-to-revise answers
- [x] Concepts, architecture diagrams & examples
- [x] Real-world use cases & industry practices
- [x] Covers LLMs, retrieval, vector DBs, evaluation, deployment, security and more
- [x] Perfect for interviews & quick revision

Who is this for?

- [x] Students preparing for internships & placements
- [x] Software engineers & ML engineers
- [x] Data/AI practitioners werking on RAG applications
- [x] Anyone preparing for system design & LLM interviews

Tools & Technologies Covered

LLMs	LangChain	LlamaIndex	OpenAI
Hugging Face	Pinecone	FAISS	Weaviate
Chroma	Milvus	Qdrant	PostgreSQL

"Better Retrieval → Better Context → Better Answers"

Insta ID: @techyy_bandaa

SUBTOPIC: RAG Fundamentals (Q1 - Q15)

Q1. What is Retrieval-Augmented Generation (RAG)?

- RAG is a technique that combines information retrieval with large language models (LLMs) to generate accurate, grounded and up-to-date answers using external knowledge sources.
- It retrieves relevant documents from a knowledge base and uses them as context for the LLM to generate a response.

Key Idea: Retrieve → Augment → Generate
(External Knowledge + LLM)

Q2. Why was RAG introduced, and what problem does it solve?

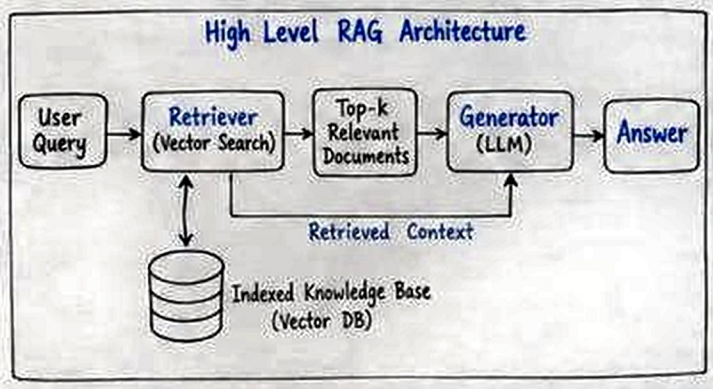
- LLMs are trained on static data and can:
  - Produce incorrect or outdated information
  - Hallucinate facts
  - Lack domain-specific knowledge
- RAG solves these problems by:
  - Providing real-time, up-to-date information
  - Grounding answers in actual documents
  - Improving factual accuracy and transparency.

Q3. How does a RAG pipeline work from query to final answer?

1. User asks a query.
2. Query is converted to an embedding.
3. Retriever searches the knowledge base for relevant documents.
4. Top-k documents are retrieved.
5. Retrieved documents are added to the prompt (context).
6. LLM generates an answer using the query + retrieved context.
7. Final grounded answer is returned to the user.

Q4. What are the main components of a RAG architecture?

- Knowledge base (documents/data source)
- Document processing (chunking, embedding, indexing)
- Retriever (searches relevant documents)
- Generator (LLM that generates final answer)
- Optional: Reranker, Query rewriter, Memory, Evaluation



Q5. What is the difference between a retriever and a generator?

	Retriever	Generator
Purpose	Finds relevant documents from knowledge base	Generates final answer using retrieved context
Input	Query (and embeddings)	Query + retrieved documents
Output	Top-k relevant documents	Natural language response
Model type	Embedding model + vector search	LLM (e.g., GPT, Llama)

Q6. What role does the knowledge base play in RAG?

- Stores the external information (documents, FAQs, APIs, DBs, etc.)
- Provides the data from which relevant context is retrieved.
- Can be static (manual documents) or dynamic (real-time data).
- Quality, freshness and structure of knowledge base directly impact RAG performance.

Q7. How does RAG reduce hallucinations in large language models?

- By providing real, relevant documents as context, the LLM bases its answer on actual information instead of relying only on its internal knowledge.
- The model is instructed to use the retrieved context, which reduces invention of facts and improves factual consistency.

Q8. What is the difference between retrieval and generation?

	Retrieval	Generation
Goal	Find relevant information	Create a natural language answer
Input	Query	Query + retrieved documents
Output	Relevant documents	Final response
Model	Embedding model + search	LLM (e.g., GPT, Llama)

Q9. What are the advantages of using RAG?

- More accurate and grounded answers
- Access to up-to-date and domain-specific information
- Reduces hallucinations
- Transparent (can show sources)
- Works well for enterprise and private data
- Cost-effective compared to fine-tuning in many cases

Q10. What are the major limitations of RAG?

- Quality depends on retrieval results (garbage in → garbage out)
- Can retrieve irrelevant docs
- Chunking and indexing challenges
- Latency (retrieval + generation)
- Not good for highly creative tasks
- Needs good evaluation and tuning

Q11. What types of applications benefit most from RAG?

- Enterprise chatbots (internal documents)
- Customer support systems
- Technical documentation Q&A
- Research and academic tools
- Legal, medical, financial information systems
- Any domain requiring accurate, up-to-date, or private information

Q12. What is the difference between traditional search and RAG?

	Traditional Search	RAG
Output	List of links/documents	Natural language answer
User effort	User reads and finds info	Direct answer with context
Technology	Keyword/BM25 search	Vector search + LLM
Use case	Information discovery	Conversational, task-oriented Q&A

Q13. What is grounding in the context of RAG?

- Grounding means the LLM's answer is based on retrieved documents from external sources.
- A grounded answer has factual support from real documents and can be traced back to the source.
- It increases trust, accuracy and transparency.

Q14. What happens when a RAG system retrieves irrelevant documents?

- The LLM may generate incorrect, off-topic or confused answers.
- Answer quality degrades significantly.
- Can still hallucinate if context is not relevant.
- Mitigation: Use better chunking, embedding models, rerankers, query rewriting, filters and relevance thresholds.

Q15. Explain a basic RAG architecture using a real-world example.

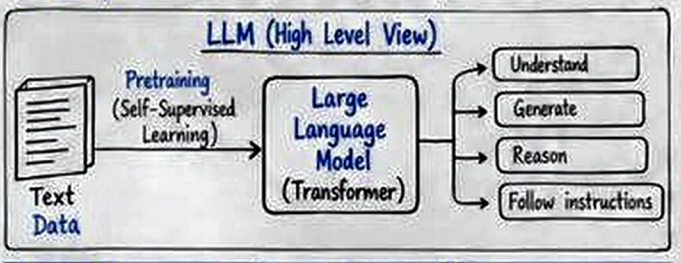
Example: Company HR Bot

Insta ID: @techyy_bandaa

SUBTOPIC: LLMs, Hallucinations & RAG vs Fine-Tuning (Q16 - Q30)

Q16. What is a large language model (LLM)?

- An LLM is a deep learning model (usually based on Transformer architecture) trained on massive amounts of text data to understand, generate and reason about human language.
- Learns patterns, grammar, facts and reasoning from data.
- Examples: GPT, Llama, Claude, Gemini, Mistral etc.



Q17. What is an LLM's knowledge cutoff?

- The knowledge cutoff is the last date of data used to train the model. Events after this date are not known to the model unless provided externally (e.g., via RAG).
- Example: If a model has a cutoff of Apr 2024, it may not know events after that.

Q18. Why do LLMs hallucinate?

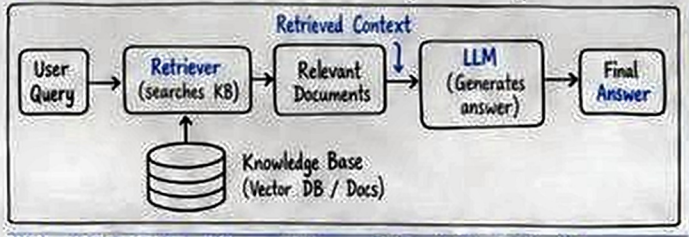
- LLMs predict the next token based on learned patterns, not on a verifiable database.
- Reasons for hallucinations:
  - Gaps or outdated knowledge
  - Ambiguous or incomplete prompts
  - Overgeneralization from training data
  - High confidence generation even when uncertain
  - No direct access to real-time or authoritative sources

Q19. What is parametric knowledge in an LLM?

- Parametric knowledge refers to the information stored inside the model's weights (parameters) after training.
- It contains generalized facts, patterns and relationships learned from training data.
- It is static and cannot be updated without re-training or fine-tuning.

Q20. How does RAG provide external knowledge to an LLM?

- RAG retrieves relevant documents from an external knowledge base (vector DB, search, etc.) based on the user query.
- These retrieved documents are added to the prompt (context) and given to the LLM.
- The LLM uses this external context to generate a grounded answer.



Q21. What is the difference between RAG and fine-tuning?

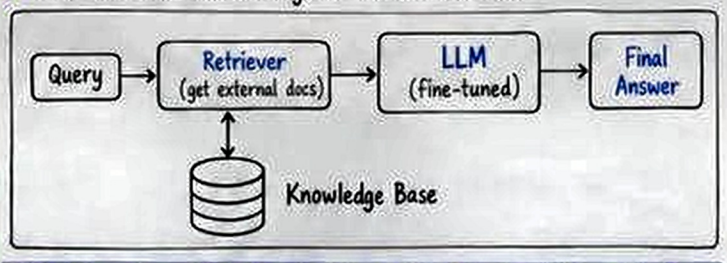
Aspect	RAG	Fine-tuning
Goal	Use external knowledge at inference	Update model behavior via training
Knowledge source	External documents	Learned into model weights
Update method	Change documents (no training)	Requires re-training / fine-tuning
Cost	Lower (no training)	Higher (training compute)
Flexibility	High (dynamic knowledge)	Lower (static after training)
Use case	Up-to-date, factual information	Domain-specific behavior/style

Q22. When should you choose RAG over fine-tuning?

- Choose RAG when:
  - Knowledge frequently changes (e.g., news, company data)
  - You need factual, up-to-date information
  - You want to avoid costly training
  - You need transparency with source documents
  - Domain knowledge is large and evolving
- Fine-tuning is better when you need:
  - Specific writing style or tone
  - Domain-specific skills or reasoning behavior
  - Better task performance on structured/generative tasks

Q23. Can RAG and fine-tuning be used together?

- Yes. They are complementary.
- Fine-tune the model for better domain understanding, format following, or task behavior.
- Use RAG to provide up-to-date and factual external knowledge.
- This combination often gives the best results.



Q24. How does RAG handle frequently changing information?

- RAG fetches the latest information from an external source (e.g., updated documents, databases, APIs, web).
- You just need to update the knowledge base, not the model.
- This allows RAG to provide current and time-sensitive answers.

Q25. What is the context window of an LLM?

- The context window is the maximum number of tokens (input + output) that the model can process at once.
- Examples:
  - GPT-3.5: ~16K tokens | GPT-4: 8K / 32K / 128K tokens
  - Claude, Gemini, etc. have larger context windows.
- Everything (query + retrieved docs + instructions + expected output) must fit within this limit.

Q26. Why not put the entire knowledge base into an LLM prompt?

- It will usually exceed the context window limit.
- It is very expensive (more tokens → higher cost).
- It introduces noisy and irrelevant information, which can degrade answer quality.
- RAG retrieves only the most relevant chunks to keep the context concise and useful.

Q27. Does RAG completely eliminate hallucinations?

- No. RAG significantly reduces hallucinations by providing relevant and factual context, but it does not completely eliminate them.
- The model can still:
  - Misinterpret retrieved information
  - Combine facts incorrectly
  - Generate unsupported claims if retrieval is poor

Q28. How does RAG differ from long-context LLM approaches?

Aspect	RAG	Long-Context LLM
How it works	Retrieve relevant docs	Use very large context window
Scalability	More scalable	Limited by context size
Cost	Lower (only relevant docs)	Higher (more tokens)
Accuracy	Better grounding	Can get distracted by noise
Use case	Large/dynamic knowledge	Small to medium knowledge

Q29. What are the cost and latency trade-offs?

Aspect	RAG	Fine-tuning
Cost	Lower (no training)	Higher (training + infra)
Latency	Slightly higher (retrieval step)	Lower (no retrieval)
Updates	Easy (update docs)	Expensive (re-train)
Inference cost	Moderate	Low per request (after training)
Total cost (long term)	Lower for changing knowledge	Higher, but better for stable tasks

Q30. How to decide between RAG, fine-tuning and prompt engineering?

1. Try prompt engineering first (for simple use cases).
2. Use RAG when you need factual, up-to-date or domain-specific knowledge.
3. Use fine-tuning when you need specific behavior, style or complex task performance.
4. Combine RAG + fine-tuning for the best results in most real-world applications.

Insta ID: @techyy_bandaa

SUBTOPIC: Data Ingestion & Preprocessing (Q31 - Q45)

Q31. What is data ingestion in a RAG pipeline?

- Data ingestion is the process of collecting, loading and preparing external data from various sources (PDFs, websites, DBs, APIs, etc.) into a knowledge base so that it can be indexed and retrieved by the RAG system.
- It converts raw data into clean, structured, searchable chunks with metadata.

Q32. What are document loaders, and why are they needed?

- Document loaders are modules that read data from different sources (PDF, TXT, HTML, DOCX, APIs, DBs, etc.) and convert them into a standard text format.
- They are needed because each data source has a different structure and parsing logic.

Common Document Loaders

- PDFLoader
- Web/HTML Loader
- DocxLoader
- CSV/JSON Loader
- Database Loader (SQL)
- API Loader

{ Converts different file types into Document objects with text + metadata

Q33. How do you extract text from PDF documents?

- Use libraries: PyPDF2, pdfminer, pypdf, or LangChain PDFLoader.
- For complex PDFs (tables, images), use layout-aware tools like PyMuPDF or Unstructured.
- Extract text page by page and preserve structure (headings, tables).

Example (PyPDF2):

```
from pypdf import PdfReader
reader = PdfReader("file.pdf")
text = ""
for page in reader.pages:
    text += page.extract_text()
```

Q34. How would you process scanned PDFs using OCR?

- Scanned PDFs contain images, so we use OCR (Optical Character Recognition) to extract text.
- Common tools: Tesseract OCR, Google Vision OCR, AWS Textract.
- Steps:
  1. Convert PDF pages to images.
  2. Apply OCR to extract text.
  3. Post-process to fix line breaks and formatting.
  4. Add metadata like page number.

Q35. How do you ingest data from websites and HTML pages?

- Use web loaders to fetch and parse HTML content.
- Common tools: BeautifulSoup, Unstructured, LangChain WebBaseLoader.
- Steps:
  1. Fetch the webpage content (handle dynamic pages if needed).
  2. Remove HTML tags, boilerplate, ads, navigation menus.
  3. Extract clean main content.
  4. Store with metadata (URL, title, last modified).

Q36. How do you clean noisy or malformed text before indexing?

- Remove unwanted content: headers, footers, page numbers, ads, HTML tags.
- Fix encoding issues and special characters.
- Normalize whitespace and line breaks.
- Remove excessive newlines, symbols, or non-text elements.
- Use regex rules and heuristics for cleaning.

Example cleaning steps:

Q39. How do you handle duplicate documents in a knowledge base?

- Duplicates reduce retrieval quality and waste storage.
- Techniques:
  - Compute document hash (e.g., SHA-256) for exact duplicates.
  - Use text similarity (embedding cosine similarity) for near-duplicates.
  - Remove or merge duplicate chunks.
  - Keep latest version based on timestamp/metadata.

Q37. What is metadata extraction in RAG?

- Metadata extraction means extracting additional information about a document or chunk, such as title, source, author, date, section, page number, URL, etc.
- It can be extracted from file properties, content or using LLMs.

Example metadata fields:

- source: "company_docs.pdf"
- page: 12
- title: "Leave Policy"
- section: "HR Policies"
- date: "2024-03-10"
- doc_type: "PDF"
- tags: ["HR", "Policy"]

Q38. Why is document metadata important for retrieval?

- Helps in filtering relevant documents (e.g., by source, date, department).
- Improves retrieval accuracy and relevance.
- Enables hybrid retrieval (semantic + metadata filters).
- Useful for traceability and citations.
- Allows domain-specific search (e.g., only HR documents).

Q40. How do you process tables, code blocks, and structured documents?

- **Tables:** Extract using layout-aware parsers, convert to markdown/CSV format, keep structure.
- **Code blocks:** Detect code sections, keep formatting, add language info.
- **Structured docs (JSON, CSV, DOCX):** Parse and convert to readable text while preserving structure.

Example representation:

Original Table

Name	Role	Dept
Aman	SDE	Engineering
Riya	HR	Human Resources

Converted to text (or markdown) →

Aman | SDE | Engineering
Riya | HR | Human Resources

Q41. What is the difference between batch and incremental ingestion?

Aspect	Batch Ingestion	Incremental Ingestion
Data size	Large (entire dataset)	Only new/updated data
Frequency	Run periodically	Continuous or scheduled
Use case	Initial load, small datasets	Frequently changing data
Cost	Higher (processes all data)	Lower (processes only changes)
Complexity	Simpler	More complex (change detection)

Q42. How would you handle updates and deletions in source documents?

- Updates: Re-ingest the document, re-chunk and update embeddings.
- Deletions: Remove from vector DB using document ID or metadata filter.
- Use versioning and timestamps to track changes.
- Maintain a mapping between source document ID and stored chunks.

Q43. What is document normalization, and why does it matter?

- Normalization means converting documents to a consistent format (e.g., encoding, text case, whitespace, date formats, units, etc.).
- It matters because it improves chunking, embedding quality and retrieval consistency across different sources.

Q44. How do you build a reliable document ingestion pipeline?

- Use modular components: loaders → parsing → cleaning → chunking → metadata extraction → indexing → monitoring.
- Handle failures, add logging and retries.
- Support multiple file types and sources.
- Validate data quality and maintain versioning.

Q45. How would you design ingestion for millions of documents?

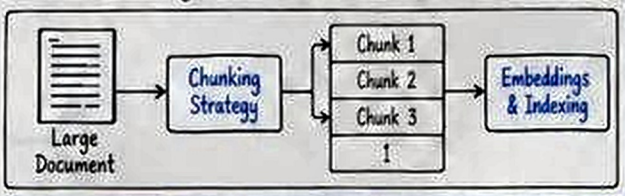
- Use distributed processing (e.g., Apache Airflow, Spark).
- Parallelize loading, parsing and embedding generation.
- Store intermediate results and use queues (e.g., Kafka, SQS).
- Use incremental processing and deduplication.
- Monitor pipeline with metrics (success rate, latency, failures).
- Use scalable storage and vector DB (e.g., Pinecone, Weaviate, Milvus).

Insta ID: @techyy_bandaa

SUBTOPIC: Chunking Strategies (Q46 - Q60)

Q46. What is chunking in RAG, and why is it necessary?

- Chunking is the process of splitting a large document into smaller, meaningful pieces (chunks) before indexing.
- A chunk typically contains a few sentences, a paragraph, or a logical section.
- It is necessary because:
  - LLMs have limited context window.
  - Smaller chunks give better retrieval accuracy.
  - Improves semantic search and reduces noise.
  - Allows fine-grained retrieval of relevant information.



Q47. What is fixed-size chunking?

- Splits text into chunks of a fixed number of characters or tokens.
- Simple and fast to implement.
- May break sentences or logical structure.
- Example: 500 tokens per chunk.

Example (chunk size = 50 words)

Words 1-50	Words 51-100	Words 101-150	...

Q48. What is sentence-based chunking?

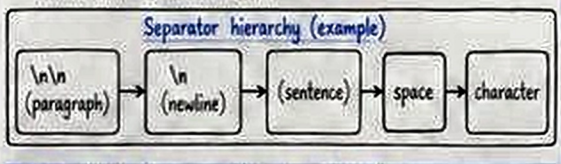
- Splits text based on sentence boundaries.
- Groups a fixed number of sentences per chunk.
- Maintains better semantic coherence than fixed-size chunking.

Example (3 sentences per chunk)

Chunk 1: Sentence 1. Sentence 2. Sentence 3.
Chunk 2: Sentence 4. Sentence 5. Sentence 6.
Chunk 3: Sentence 7. Sentence 8. Sentence 9.
...

Q49. What is recursive character text splitting?

- A strategy that tries multiple separators in a hierarchy to split text into chunks of a target size.
- Common separators: \n\n (paragraph), \n (newline), . (sentence), space, characters.
- It recursively splits on larger separators first and falls back to smaller ones.

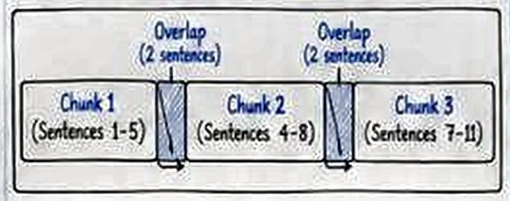


Q50. What is semantic chunking?

- Splits text based on semantic meaning rather than fixed size or sentence boundaries.
- Uses embeddings or similarity to detect natural topic boundaries.
- Creates chunks that are semantically coherent.
- Useful for long, unstructured, or domain-specific documents.

Q51. What is chunk overlap, and why is it used?

- Chunk overlap means keeping some overlapping content between consecutive chunks.
- It helps maintain context across chunk boundaries and prevents loss of important information.
- Common overlap: 10% - 30% of chunk size.



Q52. How do you choose the optimal chunk size?

- Depends on:
  - Document type (technical, conversational, legal, etc.)
  - LLM context window
  - Nature of queries
  - Granularity of information
- Common range: 200 - 1000 tokens.
- Empirical evaluation is important (try and measure).

Use case	Recommended chunk size
General text / articles	300 - 500 tokens
Technical docs / code	300 - 800 tokens
Legal / financial	200 - 400 tokens
Long reports / research papers	500 - 1000 tokens

Q53. What happens when chunks are too small?

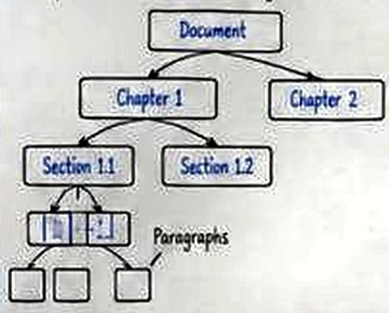
- Lack of sufficient context for retrieval.
- Important information gets split.
- More chunks to index → higher storage and retrieval cost.
- Can lead to fragmented answers and lower accuracy.

Q54. What happens when chunks are too large?

- Dilutes relevant information with irrelevant content.
- Embeddings may become less specific.
- Retrieval becomes less precise.
- May exceed context limits when sending to LLM.
- Harder to rank and filter relevant content.

Q55. What is hierarchical chunking?

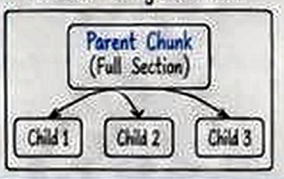
- Splits the document at multiple levels of granularity.
- Example: chapter → section → paragraph → sentence.
- Helps in both coarse and fine-grained retrieval.



Advantages	Challenges
• More meaningful chunks • Better retrieval relevance • Preserves context and topic continuity	• More complex implementation • Higher computational cost • Requires embedding model

Q56. What is parent-child chunking?

- Split documents into larger parent chunks and smaller child chunks.
- Child chunks are used for retrieval, but the parent chunk (with more context) is passed to the LLM.
- Useful when small chunks retrieve well but need more context for generation.



Q57. What is document-aware chunking?

- Uses the document structure (headings, sections, paragraphs) to create meaningful chunks.
- Preserves semantic and structural boundaries.
- Works well for structured documents like PDFs, docs, web pages, etc.

Examples of boundaries:

- Headings (H1, H2, H3)
- Sections / chapters
- Paragraphs
- Lists and tables

Q58. How do you chunk tables, source code, and technical documentation?

- **Tables:**
  - Keep table structure, convert to markdown/CSV.
  - Treat each table as a separate chunk or split by rows if very large.
- **Source code:**
  - Split by functions, classes, or logical blocks.
  - Keep related code + comments together.
- **Technical documentation:**
  - Use headings, sections, code blocks, and lists.
  - Preserve structure and metadata (title, section).

Q59. How can chunk boundaries cause retrieval failures?

- Important information may be split across two chunks.
- A chunk may contain only partial context, making it less relevant.
- Key entities, definitions, or code may be separated.
- Can lead to irrelevant retrieval or incomplete answers.
- **Solution:** use overlap, semantic chunking, or parent-child chunking.

Q60. How would you evaluate and optimize a chunking strategy?

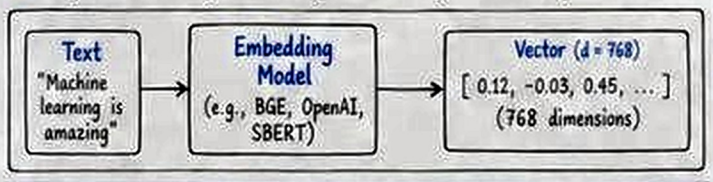
- **Evaluation metrics:**
  - Retrieval accuracy (Recall@k, MRR, nDCG)
  - Answer quality (human evaluation or LLM judge)
  - Chunk coherence (semantic similarity)
- **Optimization techniques:**
  - Try different chunk sizes and overlap values
  - Compare chunking methods (fixed, semantic, hierarchical)
  - Analyze failed queries and adjust strategy
  - Use real user queries for evaluation

Insta ID: @techyy_bandaa

SUBTOPIC: Embeddings & Semantic Search (Q61 - Q75)

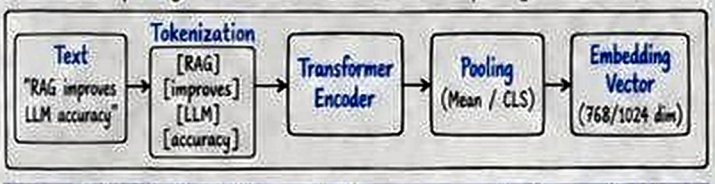
Q61. What are embeddings in the context of RAG?

- Embeddings are numerical vector representations of text (words, sentences, passages or documents) that capture semantic meaning.
- Similar meaning → vectors are close in the embedding space.
- Used in RAG to convert both queries and documents into vectors for semantic search and retrieval.
- They allow similarity search beyond exact keyword matching.



Q62. How does a text embedding model convert text into vectors?

- Uses transformer models (e.g., BERT, RoBERTa) trained with contrastive or masked language modeling objectives.
- Text is tokenized → converted to token embeddings → processed through transformer layers → pooled into a fixed-size vector.
- Common pooling methods: [CLS] token, mean pooling, or last token.



Q63. What is the difference between sparse and dense embeddings?

Aspect	Sparse Embeddings	Dense Embeddings
Representation	High-dimensional, mostly zeros	Low-dimensional, continuous
Examples	BM25, TF-IDF, SPLADE	BERT, SBERT, OpenAI, BGE
Meaning	Based on exact keyword matching	Captures semantic meaning
Dimensionality	Very high (10K - 1M)	Lower (e.g., 384, 768, 1024)
Best for	Exact term matching	Semantic similarity & intent
Use case	Lexical search	Semantic search (RAG)

Q64. What is cosine similarity, and how is it used in retrieval?

- Cosine similarity measures the cosine of the angle between two vectors.
- Values range from -1 to 1 (for text embeddings usually 0 to 1).
- Used to find most similar documents to the query in vector search.

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|} = \frac{\sum_i A_i B_i}{\sqrt{\sum_i A_i^2} \sqrt{\sum_i B_i^2}}$$

$\cos(\theta)$ close to 1 → high similarity
$\cos(\theta)$ close to 0 → low similarity

Q65. What is the difference between cosine similarity and dot product?

Aspect	Cosine Similarity	Dot Product
Formula	$\frac{A \cdot B}{\|A\| \, \|B\|}$	$A \cdot B = \sum_i A_i B_i$
Range	-1 to 1	Unbounded
Normalization	Normalized (magnitude ignored)	Not normalized
Effect of magnitude	Independent of vector length	Depends on vector length
Use in retrieval	Most common for semantic search	Can work, but sensitive to vector magnitude

Q66. What is Euclidean distance in vector search?

- Measures straight-line distance between two vectors.
- Used as an alternative similarity metric (especially in some vector DBs).

$$d(A, B) = \sqrt{\sum_i (A_i - B_i)^2}$$

Smaller distance → more similar.
Sensitive to vector magnitude.
Less commonly used than cosine similarity for text embeddings.

Q67. How do you choose an embedding model for a RAG application?

- Consider the following factors:
  - Accuracy on your domain and language
  - Supported languages (multilingual or not)
  - Embedding dimension and model size
  - Performance (speed and cost)
  - Open-source vs proprietary
  - Availability of domain-specific or fine-tuned models

Popular Models:

- OpenAI text-embedding-3 (1536 dim)
- BGE (384/768/1024 dim)
- E5 (multilingual)
- SBERT (lightweight)
- Instructor, Cohere embeddings
- Domain-specific models (biomedical, legal, code, etc.)

Q68. What is embedding dimensionality, and why does it matter?

- Dimensionality is the size of the embedding vector (e.g., 384, 768, 1024, 1536).
- Higher dimensions can capture more nuanced meaning but:
  - Increase storage cost
  - Higher computation cost for similarity search
  - May require more data to train effectively
- Lower dimensions are faster and cheaper but may lose some semantic details.

Q69. Why should query and document embeddings use compatible models?

- Both query and document embeddings must be from the same model (or compatible family) so that they are in the same vector space.
- If different models are used, vector representations are not aligned, leading to poor similarity scores and irrelevant results.

Q70. What happens if you change the embedding model after indexing documents?

- Existing document embeddings become incompatible with new query embeddings.
- You need to re-embed and re-index all documents using the new model.
- Otherwise, retrieval quality will degrade significantly.
- This can be time-consuming and may require migration of the vector database.

Q71. What is semantic search, and how does it differ from keyword search?

- Semantic search uses vector embeddings to find meaningfully similar documents.
- Keyword search (e.g., BM25) matches exact terms or keywords.

Aspect	Keyword Search	Semantic Search
Matching	Exact word/term matching	Meaning-based matching
Handles synonyms	No	Yes
Understands context	No	Yes
Works for natural language	Limited	Good
Example query	"neural network tutorial"	"how to build deep learning model"

Q72. How do multilingual embedding models work?

- Trained on multilingual datasets using contrastive learning.
- Map semantically similar texts from different languages to nearby vectors in the same embedding space.
- Examples: LaBSE, mBERT, E5-multilingual, multilingual BGE.
- Useful for cross-lingual retrieval (e.g., English query → Hindi document).

Q73. What are the limitations of embedding-based retrieval?

- May miss exact keyword matches or rare terms.
- Struggles with very long or complex queries.
- Can retrieve semantically similar but incorrect/irrelevant content.
- Sensitive to domain shift (out-of-domain data).
- May not capture fine-grained factual distinctions.

Q74. How can you improve embedding quality for domain-specific data?

- Use domain-specific or fine-tuned embedding models.
- Apply data augmentation (domain terminology, synonyms).
- Use better chunking and cleaning to provide high-quality input text.
- Combine with metadata filtering or hybrid search (sparse + dense).
- Evaluate on domain-specific datasets and fine-tune if needed.

Q75. How would you debug a semantic search system that returns irrelevant results?

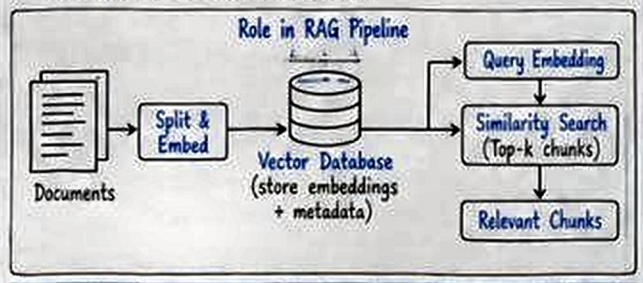
1. Check the query and retrieved embeddings (visualize or compare similarity scores).
2. Verify chunking and data quality.
3. Ensure correct embedding model and same model for query & docs.
4. Try different similarity metrics / top-k values.
5. Analyze if the query is too vague (add reformulation) or use hybrid search.
6. Inspect metadata filters and indexing pipeline for issues.

Insta ID: @techyy_bandaa

SUBTOPIC: Vector Databases & Indexing (Q76 - Q90)

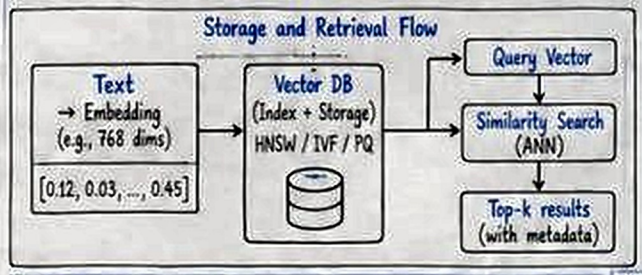
Q76. What is a vector database, and why is it used in RAG?

- A vector database is a specialized database designed to store, index and search high-dimensional vector embeddings efficiently.
- It is used in RAG to store document embeddings and retrieve the most relevant chunks for a given query using similarity search.
- Optimized for approximate nearest neighbor (ANN) search, metadata filtering and large-scale vector storage.



Q77. How does a vector database store and retrieve embeddings?

- Stores embeddings as high-dimensional vectors (e.g., 768 dims).
- Uses indexing algorithms (HNSW, IVF, PQ, etc.) for efficient search.
- On query, it computes similarity between query vector and stored vectors and returns top-k most similar vectors along with metadata.



Q78. What is the difference between a vector database and a traditional relational database?

Aspect	Vector Database	Traditional Relational DB
Data type	Vectors (embeddings)	Structured data (rows/columns)
Query type	Similarity search (ANN)	Exact match, SQL queries
Indexing	HNSW, IVF, PQ, etc.	B-tree, Hash, etc.
Use case	Semantic search, RAG	Transactional data, analytics
Schema	Flexible (vector + metadata)	Fixed schema (tables)
Strength	Similarity search at scale	ACID, complex joins, reporting

Q79. What is an approximate nearest neighbor (ANN) search?

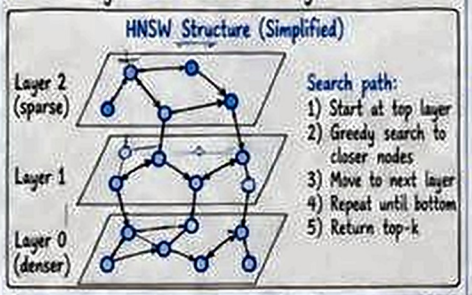
- ANN search finds vectors that are close to the query vector without exhaustive comparison with all vectors.
- It trades a small amount of accuracy for much higher speed and lower computational cost.
- Used in large-scale vector databases.

Q80. What is the difference between exact and approximate nearest neighbor search?

Aspect	Exact Search	Approximate Search (ANN)
Method	Compares with all vectors	Uses indexing (HNSW, IVF, etc.)
Accuracy	100% (true nearest neighbors)	High (but may miss some)
Speed	Slow (O(N))	Fast (sublinear)
Computational cost	High	Low
Use case	Small datasets, high accuracy	Large datasets, production systems

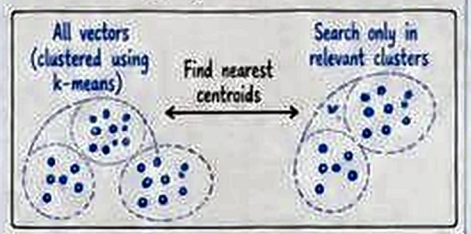
Q81. How does the HNSW indexing algorithm work?

- HNSW (Hierarchical Navigable Small World) builds a multi-layer graph of vectors.
- Each vector is connected to a few nearest neighbors.
- Search starts from top layer (sparse) and gradually moves to lower layers (denser) to find nearest vectors.
- Gives high recall with low latency.



Q82. What is IVF indexing, and when is it useful?

- IVF (Inverted File Index) clusters vectors into multiple groups (centroids) using k-means.
- During search, it first finds the nearest clusters and searches only within those clusters.
- Useful for very large datasets (millions of vectors).



Q83. What is FAISS, and how does it differ from a hosted vector database?

- FAISS (by Meta) is an open-source library for efficient similarity search and clustering of dense vectors.
- Runs locally, giving full control but you manage scaling, persistence and infrastructure.
- Hosted vector databases (Pinecone, Weaviate, Chroma, Qdrant, etc.) provide managed infrastructure, APIs, scaling, persistence, metadata filtering and additional features.

Aspect	FAISS	Hosted Vector DB
Type	Open-source library	Managed service (cloud)
Deployment	Local / self-hosted	Cloud (SaaS or self-hosted)
Scaling	Manual	Automatic
Persistence	Manual	Built-in
Metadata filtering	Limited (custom)	Built-in and advanced
Best for	Research, small/medium scale	Production, large scale

Q84. How do Pinecone, Weaviate, Chroma, and Qdrant differ?

Aspect	Pinecone	Weaviate	Chroma	Qdrant
Type	Managed (SaaS)	Open-source / Cloud	Open-source	Open-source / Cloud
Ease of use	Very easy	Moderate	Very easy	Moderate
Metadata filtering	Yes	Yes (GraphQL)	Yes	Yes
Special features	Scalable, high availability	Hybrid search, semantic + KG	Simple, lightweight	Fast, payload filtering, quantization
Best for	Production, scale	Enterprise, hybrid search	Prototyping, small-medium	Production, high performance

Q85. What is metadata filtering in a vector database?

- Allows filtering search results based on metadata fields (e.g., source, category, date, user, document type) along with vector similarity.
- Helps retrieve more relevant and domain-specific results.
- Example: search only in documents where {"source": "policy", "year": 2024}.

Q86. How do you persist and reload a vector index?

- Most vector databases persist data to disk automatically.
- For libraries like FAISS:
  - Save index using faiss.write_index(index, "index.faiss")
  - Load using faiss.read_index("index.faiss")
- Also persist metadata (id, text, other fields) separately (e.g., JSON, DB, or docstore).
- In hosted DBs (Pinecone, Weaviate etc.), persistence is managed by their infrastructure.

Q87. How do you update or delete vectors when documents change?

- Use document IDs to track vectors.
- For updates: re-embed the new content and upsert (insert or replace).
- For deletions: delete by ID.
- Some DBs support soft deletes or TTL.
- Maintain mapping between document ID and vector ID.

Q88. How do you choose the right vector database for a project?

- Consider:
  - Data size (thousands vs millions)
  - Latency and recall requirements
  - Metadata filtering needs
  - Deployment preference (self-hosted or managed)
  - Cost and scalability
  - Ecosystem and integrations
  - Special features (hybrid search, multi-tenancy, real-time updates)

Q89. How do index parameters affect recall, latency, and memory usage?

Parameter	Effect on Recall	Effect on Latency	Effect on Memory
HNSW: M (neighbors)	Higher M → higher recall	Higher M → higher latency	Higher M → more memory
HNSW: efSearch	Higher efS → higher recall	Higher efS → higher latency	Higher efS → more memory (at query time)
IVF: nlist	More clusters → better recall	May increase latency	More memory
IVF: nprobe	Higher nprobe → higher recall	Higher latency	More computation
Vector dimensionality	Higher dims → potentially better recall	Higher latency	More memory
Quantization (PQ)	Slight drop in recall	Lower latency	Lower memory

Q90. How would you design vector storage for a large enterprise knowledge base?

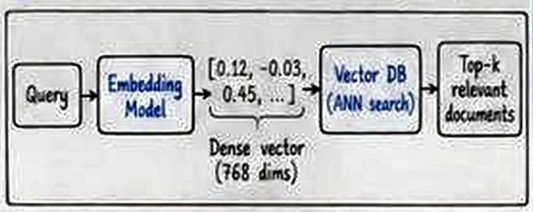
1. Choose a scalable vector database (e.g., Pinecone, Weaviate, Qdrant).
2. Use a good embedding model and proper chunking.
3. Store embeddings with rich metadata (source, department, access control).
4. Use suitable indexing (e.g., HNSW or IVF-PQ) with tuned parameters.
5. Support incremental updates, versioning and deletions.
6. Implement metadata filtering and hybrid search (semantic + keyword).
7. Ensure high availability, backups, monitoring and cost optimization.

Insta ID: @techyy_bandaa

SUBTOPIC: Retrieval Methods & Hybrid Search (Q91 - Q105)

Q91. What is dense retrieval in RAG?

- Dense retrieval uses dense vector embeddings to find semantically similar documents.
- Converts query and documents into numerical vectors (using models like BERT, OpenAI, etc.).
- Captures meaning and context, can find relevant docs even without exact keyword match.
- Common similarity metrics: cosine similarity, dot product.



Q92. What is sparse retrieval?

- Sparse retrieval uses keyword-based representations (e.g., BM25, TF-IDF).
- Documents are represented as sparse vectors based on term frequencies.
- Works well for exact term matching, rare words, numbers, and structured text.
- Does not capture deep semantic meaning.

Representation (example)

Term	doc1	doc2	doc3
rag	2	0	1
retrieval	1	3	0
vector	0	1	1
(sparse vector with mostly zeros)			

Q93. How does BM25 ranking work?

- BM25 (Best Matching 25) is a probabilistic ranking function used in information retrieval.
- Relevance score depends on term frequency (TF), inverse document frequency (IDF) and document length.

BM25 Score Formula

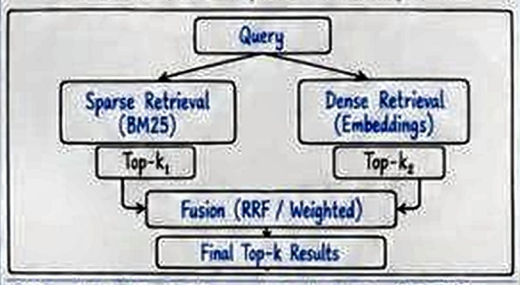
$$\text{Score}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

Where:

- $f(t,d)$ = term frequency of term t in document d
- $\text{IDF}(t)$ = inverse document frequency
- $|d|$ = length of document
- avgdl = average document length
- k_1, b = hyperparameters (commonly $k_1 = 1.2$, $b = 0.75$)

Q94. What is hybrid search, and why is it useful?

- Hybrid search combines sparse (keyword-based) and dense (semantic) retrieval to leverage both strengths.
- Useful because:
  - Keyword search is good for exact terms, numbers, and rare words.
  - Semantic search is good for meaning and context.
  - Combines both to improve recall and precision.



Q95. How do keyword search and semantic search complement each other?

Aspect	Keyword Search (Sparse)	Semantic Search (Dense)
Matching	Exact term match	Meaning-based match
Good for	Rare words, numbers, IDs, technical terms	Paraphrases, natural language, context
Representation	Sparse vectors (BM25, TF-IDF)	Dense embeddings
Strength	High precision for exact terms	High recall for semantic meaning
Limitation	Misses synonyms and rephrases	May miss exact terms and rare tokens
Together	Complementary → hybrid search gives better overall results	

Q96. What is Reciprocal Rank Fusion (RRF)?

- RRF is a simple and effective method to combine results from multiple retrieval methods (e.g., BM25 + embeddings).
- Each document gets a score based on its rank in each retrieval list.

RRF Formula

$$\text{RRF}(d) = \sum_{i=1}^{n} \frac{1}{k + \text{rank}_i(d)}$$

- $\text{rank}_i(d)$ = rank of doc d in i-th retriever
- k = constant (usually 60)
- Higher score → more relevant

Q97. How do you choose the top-k value for retrieval?

- Depends on:
  - Chunk size (smaller chunks → larger k)
  - Application type (QA, summarization, agent, etc.)
  - Model context window
  - Empirical evaluation (recall vs latency).
- Typical range: 3 - 20 (most common: 5, 10)
- Trade-off: higher k → better recall but more noise and higher latency/cost.

Example guideline:

Use case	Recommended top-k
Simple QA (short docs)	3 - 5
General QA	5 - 10
Technical / complex docs	10 - 20

Q100. How does filtering before retrieval differ from filtering after retrieval?

Aspect	Filter Before Retrieval	Filter After Retrieval
Process	Apply metadata filters while searching	Retrieve first, then filter results
Efficiency	More efficient (search smaller set)	Less efficient
Accuracy	Better (only relevant docs considered)	May miss relevant results if top-k contains irrelevant filtered docs
Use case	Structured filters (e.g., source, date)	When filter logic is complex or semantic

Q101. What is multi-vector retrieval?

- Instead of a single embedding per document, multiple embeddings are stored for different parts (e.g., title, summary, each section).
- Improves recall for long and complex documents.
- Used in models like ColBERT and multi-representation retrievers.

Q91. Q98. What is a similarity threshold, and when should you use one?

- A similarity threshold filters out low-similarity results that may be irrelevant.
- Useful when:
  - You want to avoid passing irrelevant context to the LLM.
  - Domain has sensitive or specific information.
  - Improves answer quality and reduces hallucinations.

Example:
Retrieve docs with cosine similarity score ≥ 0.7 (tune empirically).

Q99. What is metadata-based retrieval?

- Uses structured metadata (e.g., source, date, category, author, document type) to filter or boost relevant documents.
- Can be used alone or with semantic/keyword search.
- Example: retrieve only documents from "API Docs" category or from last 6 months.

Common metadata fields:

- source, author, date, category
- document type, version, language
- tags, access level, product, etc.

Q102. How would you retrieve exact product IDs or error codes that embeddings miss?

- Use keyword search (BM25) along with dense search (hybrid).
- Use regex or exact match filters.
- Store important IDs/codes in metadata or a separate keyword index.
- Use n-gram or character-level embeddings if needed.

Example approaches:

- BM25 for exact match (e.g., "ERR_404").
- Metadata filter: product_id = 'ABC123'.
- Combine with semantic search for surrounding context.

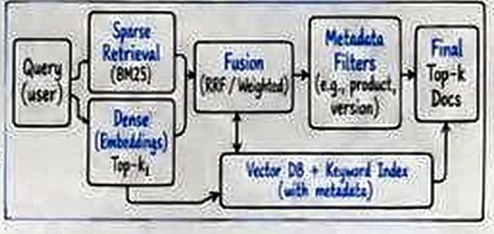
Q103. How do you handle queries containing rare technical terms?

- Use hybrid search (BM25 + dense).
- Use domain-specific embeddings or fine-tuned models.
- Keep technical glossary or synonym mapping.
- Use character/byte-level tokenization or subword models.
- Consider query expansion (add synonyms/abbreviations).

Q104. What causes low recall in a RAG retrieval system?

- Poor chunking (too large or too small).
- Inappropriate embedding model.
- Not using hybrid search for keyword-heavy queries.
- Missing metadata or incorrect filtering.
- Low top-k value.
- Domain-specific terminology not captured in embeddings.
- Poor index parameters (e.g., low ef, nprobe).
- Data ingestion issues (missing or incomplete content).

Q105. How would you design a hybrid retrieval pipeline for technical documentation?

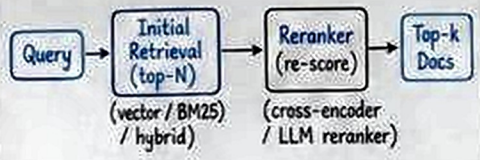


Insta ID: @techyy_bandaa

SUBTOPIC: Reranking & Retrieval Optimization (Q106 – Q120)

106. What is reranking in a RAG pipeline?

- Reranking is a second-stage process that reorders the initially retrieved documents based on a more accurate relevance model.
- Initial retrieval (e.g., vector search, BM25) brings a larger set of candidates.
- Reranker re-scores these candidates and returns a better ordered top-k list.
- Helps improve precision and overall answer quality in RAG.



107. Why might the top results from a vector database be insufficient?

- Dense retrieval captures semantic similarity but may miss exact keyword matches, numbers, product IDs, or rare terms.
- Top results can be semantically similar but not truly relevant to the user intent.
- Vector similarity ignores deeper context like query structure, negation, or specific constraints.
- Can retrieve generic documents instead of the most useful ones.

Example:

Query: "Error code 0x80070005 fix"

- Vector DB may return docs about general Windows errors, but not the exact page containing this specific code.

108. What is the difference between a bi-encoder and a cross-encoder?

Aspect	Bi-encoder	Cross-encoder
Input	Query and document encoded separately	Query and document together
Architecture	Two separate encoders (shared weights)	Single encoder with query + document as input
Similarity	Uses vector similarity (e.g., cosine)	Directly predicts relevance score
Speed	Faster (good for large retrieval)	Slower (more computational cost)
Use case	Initial retrieval (recall)	Reranking (precision)
Examples	SBERT, BGE, E5, OpenAI embeddings	BERT-based rerankers, Cohere Rerank, MonoT5

109. How does a cross-encoder reranker score query-document pairs?

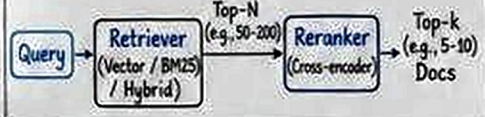
- Takes the query and document together as a single input: [CLS] query [SEP] document.
- Uses a transformer to model full interaction between tokens (attention across both).
- Outputs a relevance score (usually a single logit or probability).
- Higher score means more relevant.

Scoring process:

1. Input: query + document
2. Pass through transformer
3. Take [CLS] representation
4. Feed to a linear layer → relevance score

110. What is the retrieve-then-rerank architecture?

- Two-stage retrieval pipeline:
  1. Retrieve a larger set of candidate documents using a fast method (vector search / BM25 / hybrid).
  2. Rerank the top-N candidates using a more accurate model (e.g., cross-encoder).
  3. Return final top-k documents.



111. How many candidate documents should be retrieved before reranking?

- Typically retrieve top-N = 50 to 200 documents.
- N should be large enough to have relevant documents, but not too large (increases cost).
- Common values:
  - Small datasets: 50-100
  - Large datasets: 100-200
  - Production: 100 is a good default
- Final top-k after reranking is usually 5-10 documents.

112. How does reranking improve precision?

- Initial retrieval tends to have high recall (more documents, including noisy ones).
- Reranker uses deeper contextual understanding to identify truly relevant documents.
- Reorders the list so the most relevant documents appear at the top.
- Filters out semantically similar but not useful documents.

Result: higher precision (more relevant documents in top-k).

113. What is the difference between retrieval precision and retrieval recall?

- Precision: proportion of retrieved documents that are relevant.

$$\text{Precision} = \frac{\text{Relevant docs retrieved}}{\text{Total docs retrieved}}$$

- Recall: proportion of relevant documents that are retrieved.

$$\text{Recall} = \frac{\text{Relevant docs retrieved}}{\text{Total relevant docs in dataset}}$$

114. How do rerankers affect latency and computational cost?

- Rerankers (especially cross-encoders) are slower and more computationally expensive than bi-encoder retrieval.
- They run a transformer on each query-document pair, which is costlier.
- Latency depends on:
  - Number of candidate documents (N)
  - Model size (e.g., BERT, MonoT5)
  - Hardware (GPU/CPU)
  - Batching efficiency
- Typically adds 50ms – 500ms extra latency depending on N and model.

115. What are the limitations of cross-encoder rerankers?

- Higher latency and computational cost.
- Not scalable for very large candidate sets.
- Requires more GPU memory.
- May still miss domain-specific relevance if not fine-tuned.
- Less suitable for real-time, high-throughput applications without optimization.
- Mainly used for reranking, not for initial large-scale retrieval.

116. How do you choose a reranking model?

Consider the following factors:

- Accuracy: performance on domain-specific data
- Latency: acceptable response time
- Model size: larger models are more accurate but slower
- Domain adaptation: fine-tune if needed (e.g., technical docs)
- Availability: open-source vs closed-source
- Examples: Cohere Rerank, BGE Reranker, MonoT5, Sentence-Transformers rerankers

117. What is contextual retrieval, and how can it improve search quality?

- Contextual retrieval expands or reformulates the query using additional context.
- Techniques:
  - Query rewriting (e.g., using LLM)
  - Adding conversation history
  - Using document structure or metadata
- Helps disambiguate ambiguous queries and captures more relevant documents.
- Especially useful in multi-turn conversations.

118. How can you combine metadata filtering with reranking?

- Step 1: Apply metadata filters to narrow down the search space (e.g., product, version, date).
- Step 2: Retrieve candidate documents from the filtered subset.
- Step 3: Rerank the candidates using a cross-encoder.
- This ensures domain and structured constraints are met, while reranking improves relevance within the filtered set.

119. How do you diagnose a reranker that pushes relevant documents down the list?

- Check if the reranker is domain-mismatched (may need domain fine-tuning).
- Analyze score distribution, (are relevant docs getting low scores?).
- Look for issues:
  - Poor training data
  - Query-document format mismatch
  - Missing query context
  - Overly generic model
- Try qualitative analysis: manually inspect top-ranked vs truly relevant docs.

120. How would you optimize retrieval quality without significantly increasing latency?

- Use hybrid retrieval (BM25 + dense) to improve initial recall.
- Retrieve a moderate number of candidates (e.g., 50-100) instead of very large N.
- Use a lightweight reranker or distilled model.
- Apply metadata filtering to reduce candidate set.
- Use query rewriting or contextual retrieval to get better initial results.
- Cache frequent queries.
- Use batching and efficient inference (e.g., ONNX, quantization) for faster reranking.